

Análise de Algoritmos I

Buscamos formas de comparar algoritmos de forma independente de Hardware, Linguagem de Programação e Habilidade do Programador. Comparamos algoritmos, e não programas.

Notações

A função $T(n)$ é o custo real.

- $T(n) = \mathcal{O}(f(n)) \Rightarrow T(n) \leq c \cdot f(n)$, p/ $n > n_0$ - Upper Bound. Maior ou igual ao custo real da função. ($f(n) = n^2$, por ex)
- $T(n) = \Omega(f(n)) \Rightarrow T(n) \geq c \cdot f(n)$, p/ $n > n_0$ - Lower Bound. Menor ou igual ao custo real da função.
- $T(n) = \Theta(f(n)) \Rightarrow \geq c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$, p/ $n > n_0$ - Intermediário.

Na função $f(n)$ não se costuma incluir constantes ou termos de menor ordem.

Ex: $T(n) = 3n + 3$ (nossa função)

$$T(n) = \mathcal{O}(n)?$$

$$3n + 3 \leq c \cdot n$$

$$c \in \mathbb{N}, c \geq 4$$

Provado que $T(n)$ é $\mathcal{O}(n)$.

$$T(n) = \mathcal{O}(\log(n))?$$

$$3n + 3 \leq c \cdot \log(n)$$

Não existe c tal que satisfaça a condição acima. Provado que $T(n) \neq \mathcal{O}(\log(n))$.

$$T(n) = \Omega(n)?$$

$$3n + 3 \geq c \cdot n$$

$$c \in \mathbb{N}^*, c \leq 3$$

Provado que $T(n)$ é $\Omega(n)$.

$$T(n) = \mathcal{O}(\log(n))?$$

$$3n + 3 \geq c \cdot \log(n)$$

$$c \in \mathbb{N}^*$$

Provado que $T(n)$ é $\Omega(\log(n))$.

Exercício

$$f(n) = n^{1.5} \xRightarrow{/n} n^{0.5} \xRightarrow{\sim_2} n$$

$$g(n) = n \log(n) \xRightarrow{/n} \log(n) \xRightarrow{\sim_2} \log^2(n)$$

Para comparar, manipulamos de acordo a chegar nos algoritmos da tabela.

$$c \leq \log \leq \log^2 \leq n \leq n \log(n) \leq n^2 \leq n^3 \leq 2^n$$

Escolhendo $T(n)$: Melhor, pior, média

A função $T(n)$ pode ser o melhor, pior ou melhor caso.

$$T_{\text{melhor}(n)} \leq T_{\text{média}(n)} \leq T_{\text{pior}(n)}$$

Exercício: maxSubArray

Divisão e conquista:

-2 11 -4 13 -5 -2 -2 11 -4 13 | -5 -2 -2 11 | -4 13 | -5 -2 -> Retorna 11 | 13 | 0

Cálculo de complexidade para sub-rotinas recursivas

É necessário recorrer à análise de recorrência: Equação ou desigualdade que descreve uma função em termos de seu valor em entradas menores.

Ex: Fibonacci

- $f(0)=0, f(1)=1, f(i)=f(i-1)+f(i-2)$

```
sub-rotina fib(n: numérico)
início
declare aux numérico;
se  $n \leq 1$ 
    então aux ← 1
    senão aux ← fib(n-1)+fib(n-2);
retorne aux;
fim
```

Caso 1: $n = 0, n = 1 \rightarrow T(0) = T(1) = 3$

Caso 2: $n \geq 2 \rightarrow T(n) = T(n-1) + T(n-2) + 6$

$$\{T(0)=T(1)=3, T(n)=T(n-1)+T(n-2)+6 \mid n \geq 2\}$$

Método da substituição

Supõe-se (aleatoriamente ou com base em experiência) um limite superior para a função e verifica-se se ela não extrapola esse limite (uso de indução matemática)

Hipótese: $O(2^n)$

$$T(n) = O(2^n) \rightarrow T(n) \leq c \cdot 2^n, c > 0$$

Base da indução:

$$T(0) = 3 \leq c \cdot 2^0$$

$$T(1) = 3 \leq c \cdot 2^1$$

$$T(k-1) \leq c \cdot 2^{k-1}$$

$$T(k-2) \leq c \cdot 2^{k-2}$$

Mostrar que $T(k) \leq c \cdot 2^k$ substituindo:

$$T(k) = T(k-1) + T(k-2) + 6$$

$$T(k) \leq c \cdot 2^{k-1} + c \cdot 2^{k-2} + 6$$

$$T(k) \leq c \cdot 2 \cdot 2^{k-2} + c \cdot 2^{k-2} + 6$$

$$T(k) \leq c \cdot 3 \cdot 2^{k-2} + 6$$

$$T(k) \leq c \cdot \left(\frac{3}{2^2}\right) 2^k + 6$$

Necessário encontrar uma constante que satisfaça a desigualdade:

$$c \cdot \left(\frac{3}{2^2}\right) 2^k + 6 \leq c \cdot 2^k$$

$$c \cdot 2^k \cdot \left(\frac{3}{4} - 1\right) \leq -6$$

$$c \leq -\frac{6}{2^k \cdot \left(\frac{3}{4} - 1\right)}$$

$$c \geq \frac{24}{2^k}$$

Menor caso: $k = 0 \rightarrow \frac{24}{2^k} = 24$. Portanto, $\exists c \in \mathbb{R}$ t.q. $c \geq \frac{24}{2^k}$

Hipótese: $O(n^2)$

Base da indução:

$$T(0) = 3 \leq c \cdot 0^2$$

$$T(1) = 3 \leq c \cdot 1^2$$

$$T(k-1) \leq c \cdot (k-1)^2$$

$$T(k-2) \leq c \cdot (k-2)^2$$

Mostrar que $T(k) \leq c \cdot n^2$ substituindo:

$$T(k) = T(k-1) + T(k-2) + 6$$

$$T(k) \leq c \cdot (k-1)^2 + c \cdot (k-2)^2 + 6$$

$$T(k) \leq c \cdot [k^2 - 2k + 1 + k^2 - 4k + 4] + 6$$

$$T(k) \leq c \cdot [2k^2 - 6k + 5] + 6$$

Logo:

$$c \cdot [2k^2 - 6k + 5] + 6 \leq c \cdot k^2$$

$$c \cdot [-k^2 + 6k - 5] \geq -6$$

Não existe c fixo que satisfaça a desigualdade para qualquer k .

$$c \geq \frac{6}{-k^2 + 6k - 5}$$

Obs: ao dividir por um número negativo, o sentido da desigualdade inverte (continua não sendo válido)

Método da Árvore de recursão

$T(n) \rightarrow c, T(n/2) \rightarrow c, c, T(n/4) \rightarrow c, \dots, c$

Altura = $\log n + 1$

Método Mestre

Teorema: Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ definida como:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Então $T(n)$ pode ser limitado assintoticamente como:

- Se $f(n) = O(n^{\log_{b(a)} - x})$ para algum $x > 0$, então $T(n) = \Theta(n^{\log_b a})$
- Se $f(n) = \Theta(n^{\log_b a})$, então $T(n) = \Theta(n^{\log_b a} \log(n))$
- Se $f(n) = \Omega(n^{\log_{b(a)} + x})$ para algum $x > 0$, e se $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ para algum $c < 1$ e para todo n suficientemente grande, então $T(n) = \Theta(f(n))$

Ex: $T(n) = 9T\left(\frac{n}{3}\right) + n$

- $a = 9$
- $b = 3$
- $f(n) = n$
- $n^{\log_b a} = n^{\log_3 9} = n^2$

Tentamos o caso 1:

$$f(n) = O(n^{2-x})$$

$$n = O(n^{2-x})$$

$$n = O(n)$$

$$n \leq c \cdot n$$

Vemos que o primeiro caso é satisfeito. Logo:

$$T(n) = \Theta(n^2)$$

Ex:

$$T(n) = T\left(\frac{2n}{3}\right) + 1$$

- $a = 1$
- $b = \frac{3}{2}$
- $f(n) = 1$
- $n^{\log_b a} = n^0 = 1$

Tentamos o caso 1:

$$f(n) = O(n^{0-x})$$

$$n^0 \leq c \cdot n^{0-x}$$

O caso 1 não é válido.

Tentamos o caso 2:

$$f(n) = \Theta(n^0)$$

$$1 = \Theta(\dots)$$

???

Exercícios

$$1) T(n) = 4T\left(\frac{n}{2}\right) + n$$

$$a = 4, b = 2, f(n) = n, n^{\log_b a} = n^{\log_2 4} = 2$$

Caso 1:

$$n = \mathcal{O}(n^{\log_2(4)-x})$$

$$n = \mathcal{O}(n^{2-x})$$

$$n = \mathcal{O}(n) \rightarrow n \leq c \cdot n$$

Logo:

$$T(n) = \Theta(n^2)$$

$$2) T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

Caso 1:

$$n^2 = \mathcal{O}(n^{\log_2(4)-x})$$

$$n^2 = \mathcal{O}(n^{2-x})$$

Não existe $x > 0$ que satisfaça isso.

Caso 2:

$$n^2 = \Theta(n^{\log_2(4)})$$

$$n^2 = \Theta(n^2)$$

$$c_1 \cdot n^2 \leq n^2 \leq c_2 \cdot n^2$$

Isso é satisfeito. Logo:

$$T(n) = \Theta(n^2 \log(n))$$

$$3) T(n) = 4T\left(\frac{n}{2}\right) + n^3$$

Caso 3:

$$n^3 = \Omega(n^{\log_2 4+x})$$

$$n^3 = \Omega(n^{2+x})$$

$$n^3 = \Omega(n^3) \Rightarrow c \cdot n^3 \leq n^3$$

Verificando segunda condição:

$$a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$$

$$4\left(\frac{n}{2}\right)^3 \leq c \cdot n^3$$

$$\frac{1}{2} \leq c \leq 1$$

$$\text{Logo, } T(n) = \Theta(n^3)$$

$$4) \, T(n) = 2T\left(\frac{n}{2}\right) + n^3$$

$$5) \, T(n) = T\left(\frac{9n}{10}\right) + n$$